

PORTADA

Análisis y Diseño de Sistemas II

Semana 7 — Abstracción y Encapsulamiento

Universidad de Antioquia · Ingeniería de Sistemas



APERTURA

**Cambiar un campo interno...
y romper medio sistema que nunca debió enterarse**

CONTRA EJEMPLO

Una cuenta bancaria con todo público

```
class CuentaBancaria:
    def __init__(self, saldo):
        self.saldo = saldo    # atributo público

# En cualquier parte del sistema...
cuenta.saldo = cuenta.saldo - 500000
cuenta.saldo = -300000    # nadie lo impide
```

Cualquier parte del código puede leer y **modificar directamente** el saldo, sin pasar por ninguna regla de negocio. Nada impide dejarlo en negativo, nada registra por qué cambió.

TRANSICIÓN

El problema no es el campo — es quién puede tocarlo

El diseño de `CuentaBancaria` no distingue entre "cómo se guarda el saldo" y "qué operaciones válidas existen sobre él". Esa distinción es exactamente lo que estudiamos hoy: **abstracción** y **encapsulamiento**, los dos principios que la Unidad 4 pone como base antes de llegar a SOLID (semanas 8–9) y a los patrones de diseño (semanas 10–13).

CONCEPTO

Qué es abstracción

Abstracción es el proceso de modelar solo los aspectos de un objeto o proceso que son relevantes para el problema que se está resolviendo, dejando fuera (ocultando) los detalles de implementación que no importan desde afuera.

Una clase `CuentaBancaria` le presenta al resto del sistema una **interfaz simple** —"consignar", "retirar", "consultar saldo"— sin exponer cómo calcula internamente los intereses o en qué estructura guarda el historial de movimientos.

CONCEPTO

Interfaz pública vs. detalle interno

Toda clase bien diseñada tiene dos caras:

Interfaz pública	Detalle interno
Qué operaciones ofrece la clase (sus métodos)	Cómo están representados sus datos
Lo que el resto del sistema puede usar y debe respetar	Cómo calcula, valida o almacena internamente
Debe cambiar poco — otros dependen de ella	Puede cambiar libremente sin avisar a nadie

La **abstracción** es la **disciplina de diseñar esa interfaz** pensando en lo que el cliente necesita, no en cómo está construida la clase por dentro.

CONCEPTO

Qué es encapsulamiento

Encapsulamiento es el mecanismo que hace cumplir la abstracción: consiste en **ocultar el estado interno** de un objeto (sus atributos) y exponer su comportamiento únicamente a través de una **interfaz controlada** (sus métodos públicos), de modo que ningún código externo pueda modificar ese estado sin pasar por las reglas que la propia clase define.

Este principio también se conoce en la literatura clásica de diseño como **ocultamiento de información (information hiding)**, formulado por David Parnas en 1972.

CONCEPTO

Dos caras de la misma moneda

	Abstracción	Encapsulamiento
Pregunta que resuelve	¿Qué debe ver el cliente de esta clase?	¿Cómo evito que toque lo que no debe ver?
Nivel	Diseño de la interfaz	Control de acceso al estado
Resultado	Una interfaz simple y estable	Un estado interno protegido y consistente

No hay una sin la otra: una interfaz bien abstraída pierde su valor si el estado interno sigue siendo accesible por otra vía.

CONCEPTO

Por qué importan para el diseño

- **Reducen el acoplamiento:** el cliente depende de la interfaz, no de la implementación — se puede cambiar cómo funciona por dentro sin romper a nadie que la use.
- **Protegen invariantes:** reglas como "el saldo nunca puede quedar negativo" solo se pueden garantizar si el saldo no es accesible directamente desde afuera.
- **Facilitan el cambio:** cambiar la estructura de datos interna de una clase no debería obligar a modificar el resto del sistema.
- **Bajan el costo cognitivo:** quien usa la clase solo necesita entender su interfaz, no sus 200 líneas internas.

CONTRA EJEMPLO

Mal encapsulamiento: getters y setters de todo

```
class CuentaBancaria:
    def get_saldo(self): return self._saldo
    def set_saldo(self, valor): self._saldo = valor
    def get_movimientos(self): return self._movimientos
    def set_movimientos(self, lista): self._movimientos = lista
```

Poner los atributos en privado (`_saldo`) y luego exponer un `get` y un `set` para **cada** uno no protege nada — es la misma clase pública de antes, con más código y una falsa sensación de seguridad. Cualquiera puede seguir haciendo `cuenta.set_saldo(-300000)` .

CONCEPTO

La clase anémica (Anemic Domain Model)

Una **clase anémica** es aquella que solo contiene datos (atributos con sus getters/setters) y ninguna lógica de negocio — todo el comportamiento que debería vivir dentro de la clase termina disperso en clases de "servicio" externas que manipulan esos datos desde afuera.

Es un antipatrón de diseño orientado a objetos: convierte a las clases en simples contenedores de datos y traslada toda la responsabilidad de mantener reglas de negocio a quien las usa — exactamente lo contrario de lo que promete el encapsulamiento.

CONCEPTO

Buen encapsulamiento: comportamiento, no exposición

```
class CuentaBancaria:
    def __init__(self, saldo_inicial):
        self._saldo = saldo_inicial

    def retirar(self, monto):
        if monto > self._saldo:
            raise ValueError("Fondos insuficientes")
        self._saldo -= monto

    def consultar_saldo(self):
        return self._saldo
```

El estado (`_saldo`) sigue siendo privado, pero ahora se modifica **únicamente a través de operaciones que conocen las reglas del negocio**. La clase decide qué es un cambio válido — el cliente ya no puede saltárselo.

CONCEPTO · RESUMEN

Mal vs. buen encapsulamiento

Señal de mal encapsulamiento	Cómo se ve el buen encapsulamiento
Atributos públicos o con getter/setter por cada campo	Atributos privados, expuestos solo si de verdad hace falta
La clase no valida nada — cualquier valor entra	Los métodos validan antes de modificar el estado
La lógica de negocio vive afuera, en un "manager" o "helper"	La lógica vive junto a los datos que gobierna
Cambiar la estructura interna rompe código en otros archivos	Cambiar la estructura interna no afecta a quien usa la clase

CONCEPTO

Encapsulamiento más allá de una sola clase

La misma idea aplica a escalas más grandes que un objeto individual:

- **A nivel de módulo/paquete:** un paquete puede ocultar clases auxiliares y exponer solo las que forman su API pública.
- **A nivel de capa:** en la arquitectura N-capas de la semana 6, la capa de `Repository` encapsula el detalle de cómo se accede a la base de datos — el `Service` solo conoce su interfaz.

El mismo principio que protege un atributo dentro de una clase es el que protege una capa completa dentro de una arquitectura.

TENSIÓN CONCEPTUAL

Encapsulamiento estricto vs. necesidades prácticas

En teoría, expondríamos solo comportamiento y nunca datos crudos. En la práctica, muchos frameworks (serialización JSON, ORMs, formularios web) **exigen** getters y setters, o clases de datos simples (DTOs — Data Transfer Objects, objetos cuyo único propósito es transportar datos entre capas).

La salida no es eliminar esos getters/setters, sino no confundirlos con el diseño del dominio: un DTO puede ser una clase de datos simple porque su trabajo es exactamente ese; una clase de dominio con reglas de negocio no debería serlo.



INTERACCIÓN

Encuentren la violación de encapsulamiento

```
class Carrito:
    def __init__(self):
        self.items = []
        self.total = 0

# En otro archivo, en cualquier momento:
carrito.total = 999999999
carrito.items.clear()
```

En parejas, en el chat: ¿qué invariante del carrito queda sin proteger? ¿Cómo rediseñarían la clase para que `total` nunca pueda quedar inconsistente con `items` ?

5 minutos · respondan en el chat

CONCEPTO

Heurísticas prácticas al diseñar una clase

- **Empieza todo en privado** y expón solo lo que un caso de uso real necesita — nunca al revés.
- **Pregunta "¿qué puede hacer este objeto?"** antes de "¿qué datos tiene?" — el comportamiento debe guiar el diseño de la interfaz.
- **Si necesitas un setter, pregúntate qué regla de negocio debería validar** antes de aceptar el nuevo valor.
- **Si una clase solo tiene getters/setters y cero lógica, sospecha de un modelo anémico.**

CONCEPTO · LO QUE VIENE

La base de todo lo que sigue en la Unidad 4

Abstracción y encapsulamiento son el punto de partida: mañana veremos **cohesión y acoplamiento**, que miden qué tan bien diseñadas quedan las responsabilidades entre varias clases una vez que cada una encapsula lo suyo correctamente.

Más adelante, en las semanas 8 y 9, estos mismos principios reaparecen como parte formal de SOLID — hoy los usamos con intuición de diseñador, ahí les pondremos nombre y reglas precisas.

SÍNTESIS

Ideas clave de hoy

1. **Abstracción:** modelar solo lo relevante, ocultando el detalle de implementación detrás de una interfaz simple.
2. **Encapsulamiento:** ocultar el estado interno y exponerlo solo a través de una interfaz controlada — también llamado ocultamiento de información.
3. Getters y setters de **todos** los atributos no son encapsulamiento — son la misma exposición, disfrazada.
4. Una **clase anémica** traslada la lógica de negocio afuera de los datos que gobierna: es un antipatrón, no un estilo neutral.
5. El mismo principio aplica a distintas escalas: clase, módulo, capa.



CIERRE

Para la próxima sesión

Revisen el modelo de dominio de su propio proyecto (Unidad 2) y marquen cualquier clase donde encuentren atributos expuestos sin necesidad, o lógica de negocio que debería vivir dentro de una clase y hoy vive afuera.

El miércoles vemos cohesión y acoplamiento — qué tan bien repartidas están las responsabilidades entre las clases de un sistema, y por qué esa distribución es tan importante como el encapsulamiento de cada clase por separado.

